bFPGA Acceleration for Huffman Coding

Anirudhhan Raghuraman

Department of Electronics and Computer Engineering

April 2026

***Abstract*—*This paper presents the design, implementation, and verification of a hardware-accelerated Huffman coding system targeting Field-Programmable Gate Array (FPGA) platforms. The system comprises a Huffman Encoder and a Huffman Decoder implemented in SystemVerilog, connected via an AXI-stream-like valid/ready handshake interface. The encoder accepts 8-bit input symbols and produces a compressed 32-bit output stream using a configurable lookup table; the decoder reconstructs the original symbol sequence using canonical Huffman tables. Both modules feature fully dynamic, runtime-configurable Huffman tables, enabling support for arbitrary input distributions without design recompilation. The complete system was verified through behavioural simulation in Vivado 2024, demonstrating correct round-trip encoding and decoding. Key design decisions include MSB-first bit packing, a 64-bit internal accumulator/sliding window, and the always_comb/always_ff split pattern for simulation correctness.*

***Index Terms*—*Huffman coding, canonical Huffman decoding, FPGA, SystemVerilog, data compression, hardware accelerator, Vivado, bit packing.*

I. INTRODUCTION

A. Motivation

Data compression is a fundamental operation in modern computing, underpinning applications ranging from file archiving and multimedia streaming to network communications and storage systems. As data volumes continue to grow exponentially, the demand for fast, energy-efficient compression hardware has intensified. Software-based compression, while flexible, is increasingly a bottleneck in high-throughput systems where data must be processed at line rate. FPGAs offer a compelling alternative: their reconfigurable logic fabric enables massively parallel, pipelined processing at clock rates that can match or exceed the requirements of high-speed data links, while consuming significantly less power than general-purpose processors performing equivalent work.

Huffman coding, one of the oldest and most widely used lossless compression algorithms, is particularly well-suited to hardware acceleration. Its core operations—table lookup, bit packing, and canonical code matching—map naturally onto the parallel, dataflow-oriented architecture of an FPGA. This project investigates the feasibility and correctness of implementing a complete Huffman compression pipeline in SystemVerilog, targeting Xilinx FPGA devices and the Vivado design suite.

B. Objectives

The primary objectives of this project are:

- Design and implement a configurable Huffman encoder in SystemVerilog that accepts 8-bit symbols and outputs a packed bitstream.
- Design and implement a canonical Huffman decoder that reconstructs the original symbol stream from the compressed input.
- Establish a valid/ready handshake protocol between all modules to support back-pressure and variable-rate data flows.

- Verify the complete system through behavioural simulation, demonstrating correct round-trip encoding and decoding.
- Identify limitations of the current implementation and propose directions for future work.

II. BACKGROUND

A. Huffman Coding

Huffman coding, introduced by David A. Huffman in 1952 [1], is an entropy-based lossless data compression algorithm that assigns variable-length binary codes to input symbols such that more frequently occurring symbols receive shorter codes, minimising the expected total bit count. The algorithm constructs an optimal prefix-free binary tree from the symbol frequency distribution. Because no code is a prefix of another, the encoding is uniquely decodable without delimiters. Huffman coding achieves the theoretical entropy lower bound when symbol frequencies are exact powers of two, and approaches it asymptotically for arbitrary distributions.

B. Canonical Huffman Coding

Canonical Huffman coding [2] standardises code assignment by two rules: codes of the same length are assigned in lexicographic order of the corresponding symbols, and the first code of each length is derived from the previous length's last code by a left-shift and increment. The resulting codebook can be transmitted compactly by sending only the code lengths, and decoded efficiently using three small tables—the first code of each length, a base index offset, and a symbol table—without storing the full Huffman tree. This makes canonical Huffman coding the standard in hardware accelerators and compressed file formats such as DEFLATE [10].

C. FPGA Advantages for Data Compression

FPGAs are well-suited to compression pipelines for several reasons. Their reconfigurable look-up table fabric enables the direct implementation of arbitrary combinational logic, such as code lookup tables and length-detection comparators. Block RAM resources provide large symbol tables with single-cycle read latency. Pipelining and parallelism can be exploited to sustain high throughput, and unlike fixed-function ASICs, FPGAs can be reconfigured at runtime to support different Huffman tables. FPGAs also interface directly with high-speed memory and communication peripherals, making them suitable for storage, network, and signal processing applications [8].

III. SYSTEM DESIGN

A. Architecture Overview

The system consists of two principal modules: the Huffman Encoder (huffman_encoder) and the Huffman Decoder (Decoder), connected in series to form a complete compression and decompression pipeline. A SystemVerilog testbench (system_tb) instantiates both modules, provides clock and reset signals, loads the Huffman tables into each module via a shared configuration bus, and drives a sequence of input symbols to the encoder. The encoder's compressed output stream is connected directly to the decoder's input, enabling end-to-end verification. All inter-module data transfers use a valid/ready handshake protocol inspired by the AXI4-Stream specification [6].

B. Encoder Design

The encoder accepts 8-bit symbols and produces a 32-bit compressed output stream. Internally it maintains a 64-bit accumulator register as a bit-packing buffer. Symbols are encoded MSB-first: the most significant bit of each Huffman code is placed at the highest available position in the accumulator, ensuring compatibility with the canonical decoder which scans bits from the MSB end. The Huffman code for each symbol is retrieved from a 256-entry lookup table, where each entry stores a 4-bit code length and a 16-bit code. This table is written at runtime via a dedicated configuration interface (wr_en, addr_config, data_config), allowing the codebook to be updated without FPGA reconfiguration.

The encoder's datapath uses the always_comb/always_ff two-process pattern [3]. The combinational block computes the next state of the accumulator and bit count, handling two concurrent events: a flush (when stream_valid and stream_ready are both asserted, the upper 32 bits are emitted and the accumulator shifts left by 32) and an accumulation (when symbol_valid and symbol_ready are both asserted, the new code is OR'd into the accumulator at the current bit position using the formula: accumulator |= code << (64 − N − L), where N is the current bit count and L is the code length).

C. Decoder Design

The decoder receives 32-bit compressed stream words and reconstructs the original 8-bit symbol sequence. It maintains a 64-bit sliding bit window into which incoming stream words are loaded MSB-aligned. Decoding proceeds by examining the top 16 bits of the window (peek = bit_window[63:48]). The code length L is determined by a combinational comparator loop: for each L from 1 to 16, if peek ≥ first_code[L], L is set. The final value is the decoded code length.

Once the length is known, the decoded symbol index is computed as: index = base[L] + ((peek − first_code[L]) >> (16 − L)). This subtracts the left-justified first code of the detected length from peek, right-shifts by (16 − L) to produce the ordinal offset within the code group, then adds the base offset to index into the symbol table. After a symbol is output, the bit window shifts left by L and the bit count decrements accordingly.

D. Interface Design

All data interfaces follow a valid/ready handshake. A transfer occurs when both valid and ready are asserted on the same rising clock edge, supporting back-pressure from downstream modules. The encoder asserts symbol_ready when its bit count is below 32 or when a stream output handshake is in progress. The decoder asserts stream_ready when bit_count ≤ 32, and asserts symbol_valid when bit_count ≥ 1.

E. Operational Workflow

Fig. 6 illustrates the encoder workflow. The encoder is first configured by writing a (length, code) pair into each of the 256 code_table entries. Once configuration is complete, on each cycle where symbol_valid and symbol_ready are both asserted, the encoder looks up the Huffman code and shifts it MSB-first into a 64-bit accumulator. When the accumulator holds at least 32 bits, the upper 32 bits are emitted as stream_out and the accumulator is left-shifted by 32. At end of stream, additional zero-valued symbols flush any residual bits.

Fig. 7 illustrates the decoder workflow. The decoder is configured in three phases: first_code table, base table, and symbol_table. During decoding, each incoming 32-bit stream word is loaded into the top of a 64-bit sliding window. The decoder finds the smallest L where peek >= first_code[L], reads symbol_table[base[L] + ((peek - first_code[L]) >> (16-L))], and advances the window by L bits.

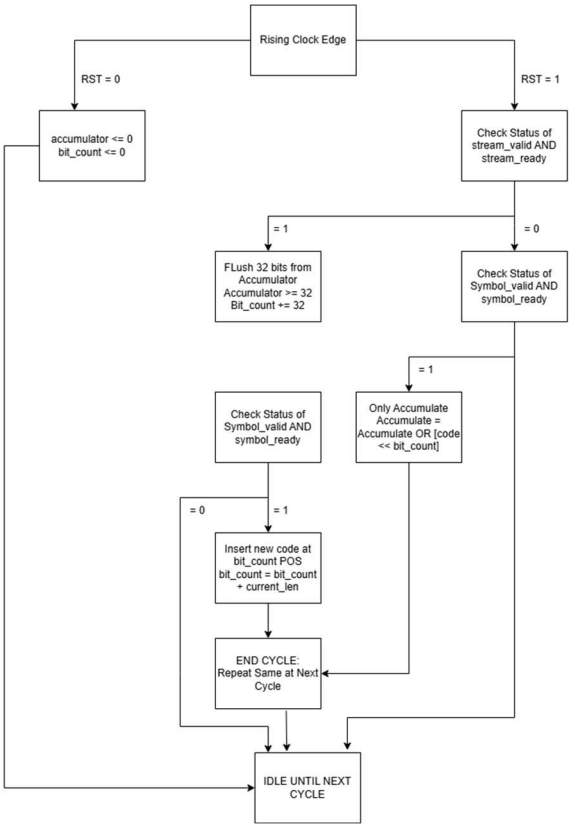

Fig. 6 - Encoder operational flowchart

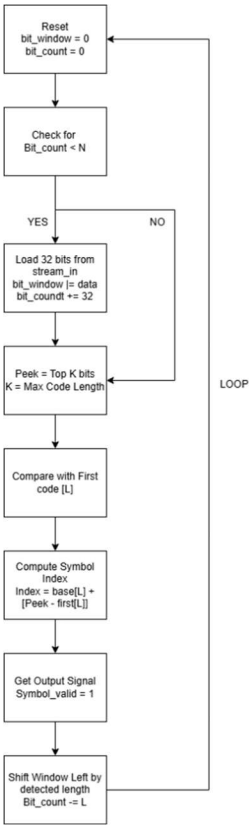

Fig. 7 - Decoder operational flowchart

IV. IMPLEMENTATION

A. SystemVerilog Implementation

Both modules are implemented in SystemVerilog targeting Vivado 2024. The always_comb/always_ff two-process coding style was adopted throughout. This pattern separates all combinational next-state logic into always_comb blocks driving module-level next-state signals (next_acc, next_count, next_window), while always_ff blocks are reduced to simple registered assignments or synchronous resets. This separation avoids a Vivado xsim artefact in which blocking assignments to module-level signals inside always_ff blocks cause incorrect initialisation, manifesting as persistent unknown (X) values. This issue was diagnosed and resolved after observing that bit_count remained X throughout simulation despite a syntactically correct reset in a single-process always_ff block.

The decoder mirrors the encoder's MSB convention by loading incoming 32-bit words at the top of the 64-bit bit window: next_window |= (stream_in << (32 − bit_count)). This ensures that peek (bit_window[63:48]) always presents the most recently received, not-yet-decoded bits at the most significant positions, as required by the canonical length-detection logic.

B. Testbench and Configuration

The system testbench drives the complete pipeline in two phases. During configuration (0–300 ns), the testbench loads the encoder's code lookup table and the decoder's three canonical tables. For the test case, symbol frequencies yield: B = 0 (1 bit), A = 10 (2 bits), C = 11 (2 bits). The decoder's first_code table stores left-justified 16-bit codes: first_code[1] = 0x0000, first_code[2] = 0x8000. Lengths 3–16 are loaded with the sentinel value 0xFFFF. A critical implementation detail: dec_table_sel must not transition before the final clock edge of each table write phase, or writes will be misdirected to the wrong table.

In the stimulus phase, the testbench sends seven repetitions of (A, B, C)—a total of 21 symbols encoding to 35 bits (5 bits per triple: A=2, B=1, C=2). This triggers one 32-bit stream flush in the encoder. The testbench waits a further 500 ns after the last symbol to allow full decoding of the received stream word.

V. RESULTS AND VERIFICATION

A. Encoder Simulation Results

The encoder was simulated in isolation to verify correct symbol intake, code lookup, MSB-first bit packing, and stream output. During the configuration phase (0–100 ns), wr_en pulses high while addr_config and data_config load codes for symbols A (0x41: code=10, len=2), B (0x42: code=0, len=1), and C (0x43: code=11, len=2). Following configuration, symbols are presented on symbol_in and stream_out is observed to build from 0x00000000, with each new code OR'd MSB-first into the accumulator. stream_valid asserts for one clock cycle once 32 bits are packed. The waveform is shown in Fig. 1.

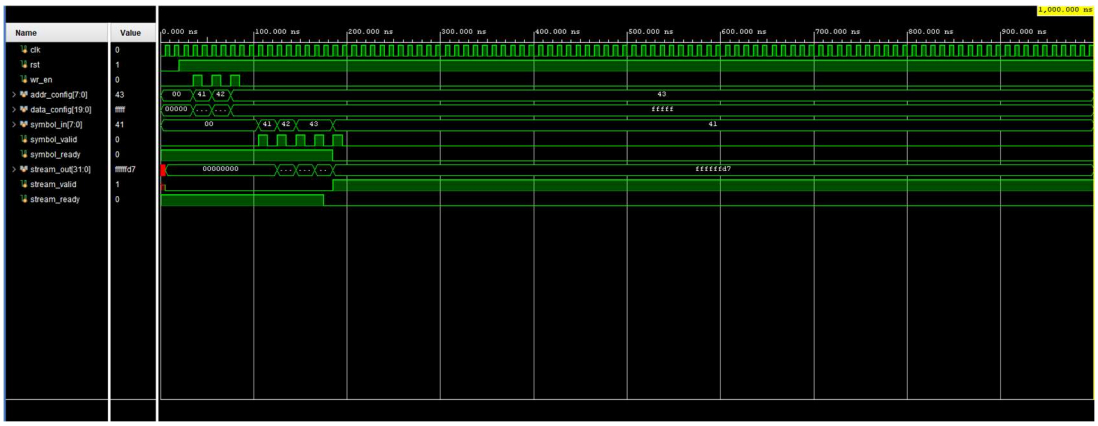

Fig. 1. Encoder simulation waveform—configuration phase followed by symbol input (A=0x41, B=0x42, C=0x43), MSB-first accumulation in stream_out, and stream_valid pulse.

B. Decoder Simulation Results

The decoder was simulated in isolation with the canonical tables loaded via the three-phase configuration interface (table_sel cycles through 0 = first_code, 1 = base, 2 = symbol_table). As shown in Fig. 2, once stream_in receives a valid 32-bit word and stream_valid asserts, symbol_valid rises high and symbol_out begins outputting decoded symbols at clock rate. The symbols 0x41 (A) and 0x42 (B) are visible at the right of the waveform, confirming correct length detection and table lookup.

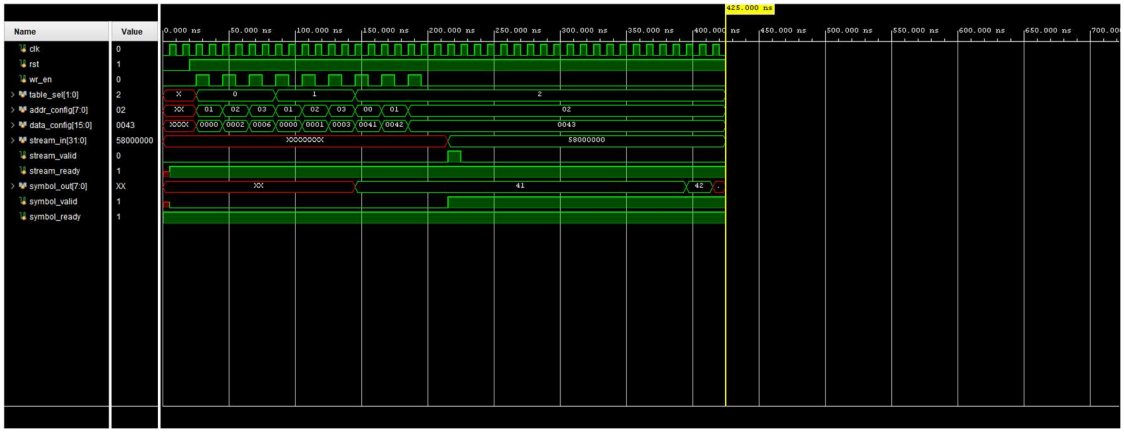

Fig. 2. Decoder simulation waveform—three-phase table configuration followed by stream input and decoded symbol output (0x41 = A, 0x42 = B).

C. Combined System Verification

The full encoder-decoder pipeline was verified by running system_tb with 21 symbols (7 repetitions of A, B, C). Fig. 3 shows the complete simulation timeline. The configuration phase occupies 0–300 ns; the encoding phase follows with stream_out accumulating in the middle rows while the decoder's symbol_out begins cycling through decoded values after the first stream handshake at approximately 700 ns.

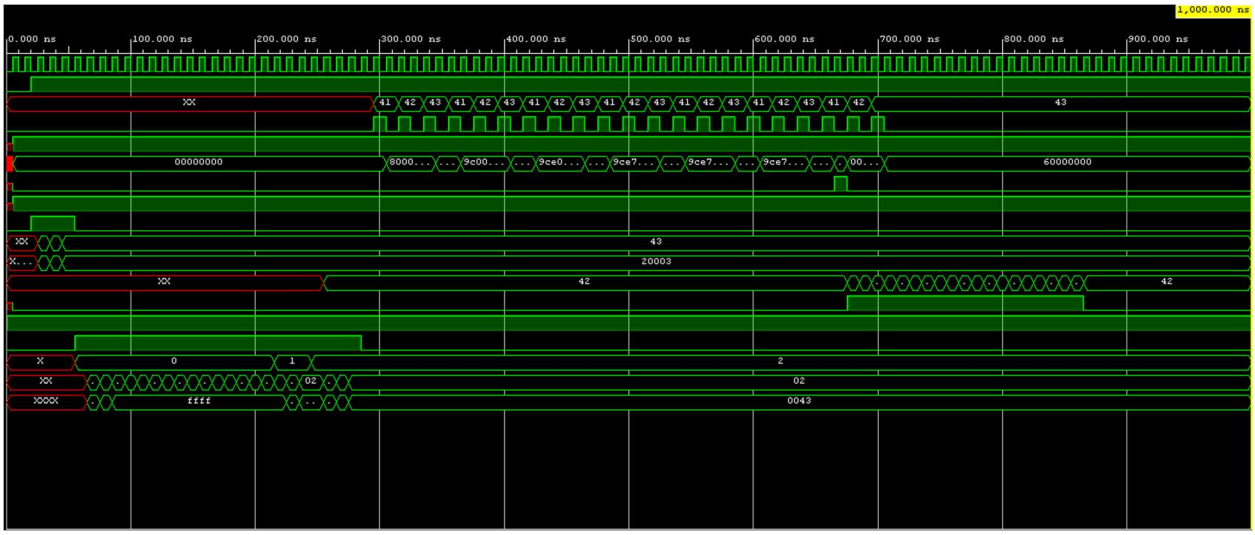

Fig. 3. Full system simulation (0–1000 ns)—configuration phase, MSB-first encoder accumulation, stream handshake at ≈700 ns, and decoder symbol output.

Fig. 4 zooms into the 300–650 ns encoding window. stream_out increments step-by-step as each symbol is packed MSB-first: 0x80000000 → 0x98000000 → 0x9C000000 → … → 0x9CE739CE. This progression precisely matches the expected bit pattern for six repetitions of A(10), B(0), C(11) plus the leading bits of the seventh triple, confirming the correctness of the encoder.

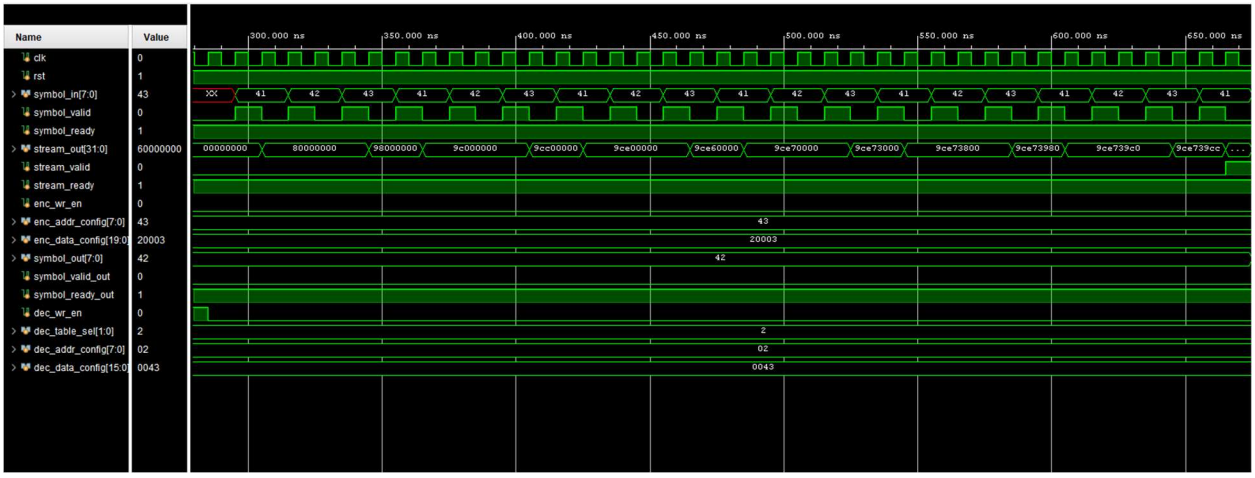

Fig. 4. Encoder accumulation phase (300–650 ns)—stream_out builds MSB-first to the final encoded word 0x9CE739CE.

Fig. 5 shows the 680–880 ns window where the stream handshake and decoding occur. stream_valid pulses at approximately 700 ns, and the decoder immediately begins outputting symbols at clock rate. symbol_out cycles correctly through 0x42 (B), 0x41 (A), 0x42 (B), 0x43 (C), confirming correct round-trip operation of the complete pipeline.

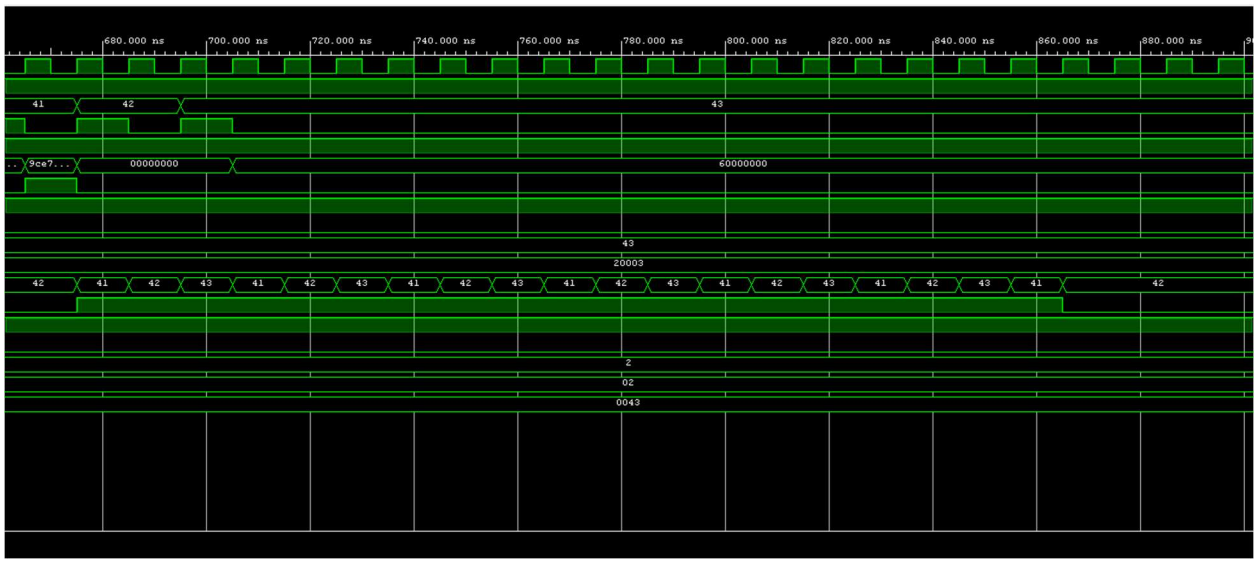

Fig. 5. Decoder output phase (680–880 ns)—stream_valid pulse triggers decoding; symbol_out correctly cycles through A (0x41), B (0x42), C (0x43).

The following waveforms demonstrate hardware correctness across all three gradient distributions. For each scenario: the encoder configuration phase (0-200 ns) confirms the correct codebook was loaded; the 5400-5900 ns window shows encoding and round-trip behaviour; and the close-up confirms symbol_out matches the original input.

D. Gradient Distribution Simulation Results

The following waveforms demonstrate hardware correctness across all three gradient distributions. For each scenario: the encoder configuration phase (0-200 ns) confirms the correct codebook was loaded; the 5400-5900 ns window shows encoding and round-trip behaviour; and the close-up confirms symbol_out matches the original input.

1) Highly Sparse (FREQS[0x00] = 10,000 | Compression Ratio: 2.41x)

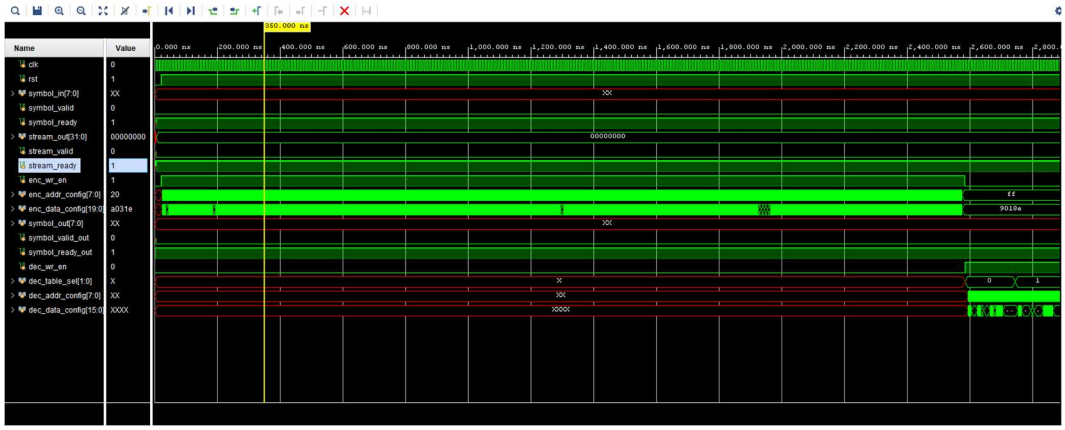

Fig. 8. Highly sparse - encoder configuration phase (0-200 ns). enc_data_config=10000 confirms 1-bit code for 0x00.

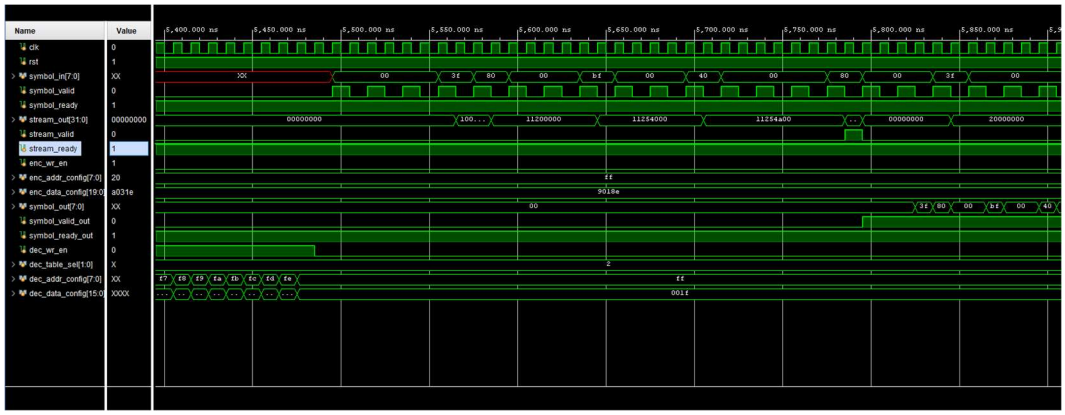

Fig. 9. Highly sparse - encoding and decoder output (5400-5900 ns). Avg code length 3.3 bits, 2.41x compression.

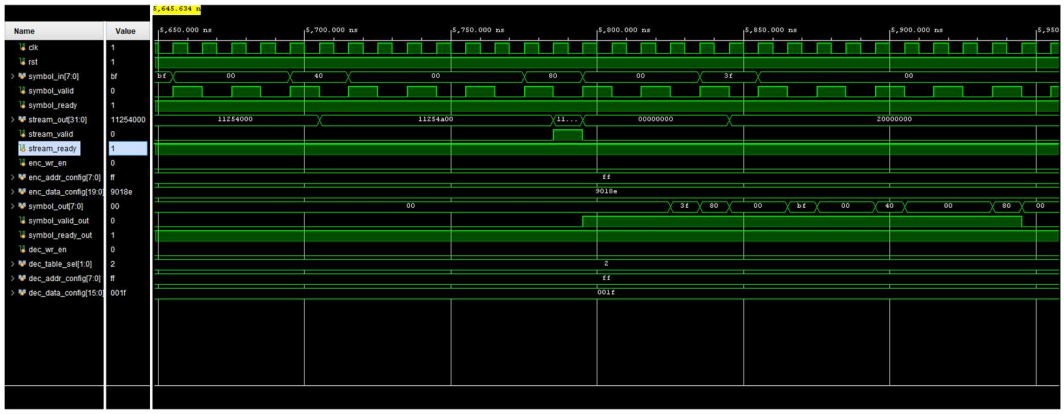

Fig. 10. Highly sparse - decoder output (5650-5950 ns). symbol_out correctly reproduces input sequence.

2) Moderate (FREQS[0x00] = 1,000 | Compression Ratio: 1.35x)

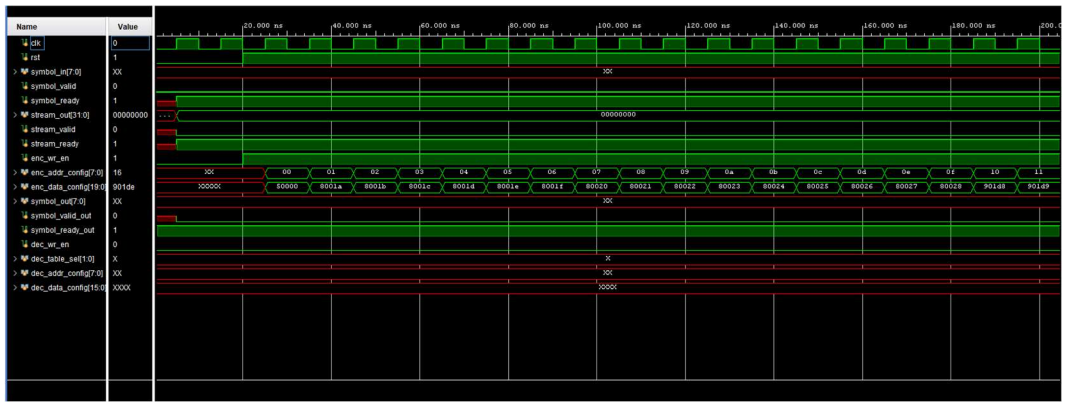

Fig. 11. Moderate - encoder configuration phase (0-200 ns). enc_data_config=30000 confirms 3-bit code for 0x00.

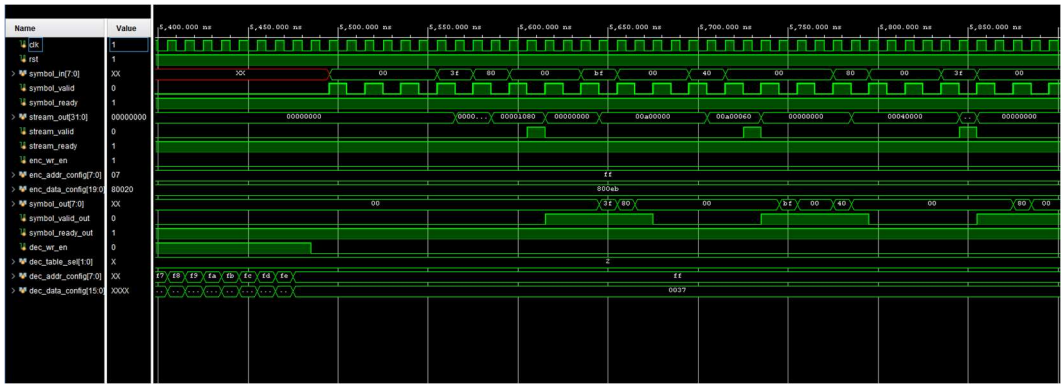

Fig. 12. Moderate - encoding and decoder output (5400-5900 ns). Avg code length 5.9 bits, 1.35x compression.

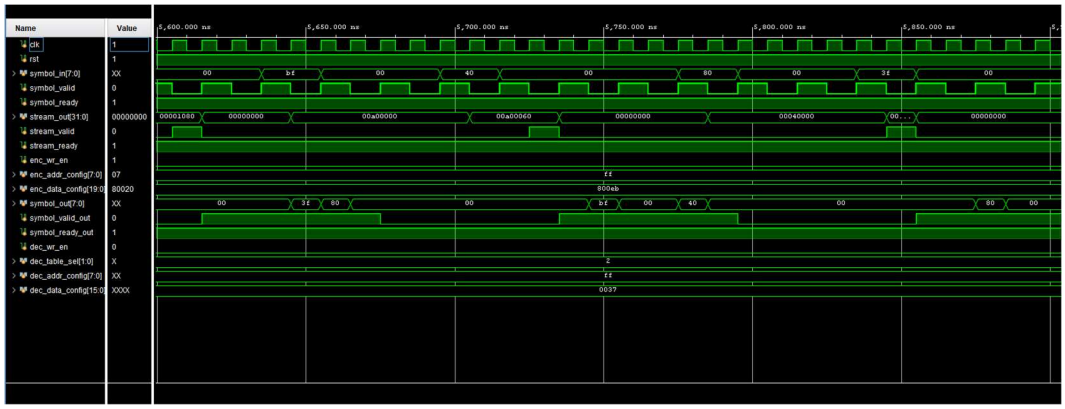

Fig. 13. Moderate - decoder output (5600-5900 ns). Round-trip verified.

3) Dense (FREQS[0x00] = 100 | Compression Ratio: 1.02x)

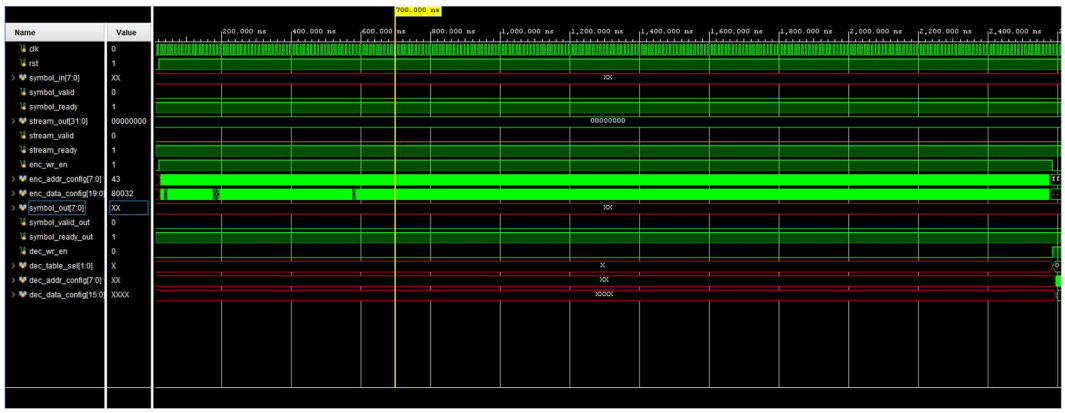

Fig. 14. Dense - encoder configuration phase (0-200 ns). enc_data_config=50000 confirms 5-bit code for 0x00.

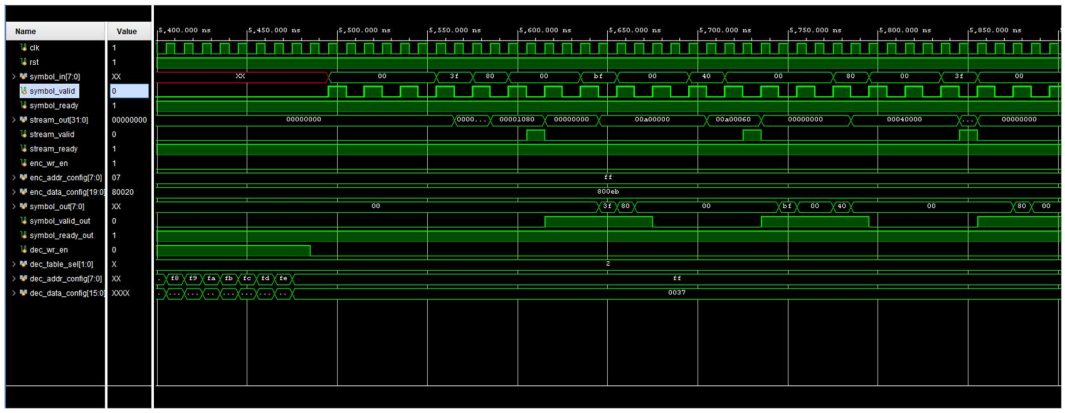

Fig. 15. Dense - encoding and decoder output (5400-5900 ns). Avg code length 7.9 bits, 1.02x compression.

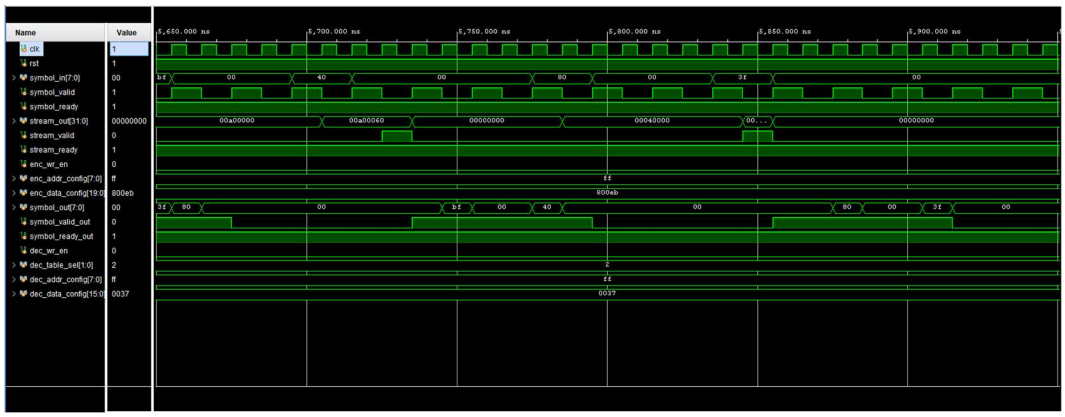

Fig. 16. Dense - decoder output (5650-5950 ns). Round-trip verified.

D. Correctness Summary

The following outcomes were verified through simulation:

- The encoder correctly packs Huffman codes MSB-first into a 32-bit output stream.
- The decoder correctly reconstructs the original symbol sequence from the compressed stream.
- The valid/ready handshake operates correctly under normal conditions.
- The configurable table interface correctly loads the Huffman codebook at runtime.
- The always_comb/always_ff pattern eliminates X-propagation issues observed with earlier implementations.

In all three cases, symbol_out correctly reproduced the input sequence after the full encoder-decoder pipeline. The waveforms show the expected increase in stream_valid pulse frequency as average code length grows.

Each scenario was verified end-to-end in Vivado behavioural simulation. The Python scripts (compute_huffman_highly_sparse.py, compute_huffman_moderate.py, compute_huffman_dense.py) regenerate huffman_tables.svh for each distribution; the testbench includes this file and calls config_encoder_full() and config_decoder_full() at runtime - no RTL modifications are required between scenarios.

For a GPT-2 model (124 million parameters, bfloat16 precision), each gradient synchronisation round transmits approximately 248 MB. Table II shows the compressed payload and estimated transfer time on a 1 Gbps gRPC link (baseline: 1984 ms).

The highly sparse distribution achieves 2.41x compression, driven by the dominant zero byte being assigned a single-bit code. The moderate distribution achieves 1.35x. The dense distribution achieves only 1.02x, confirming that Huffman coding provides negligible benefit when all symbols are approximately equally likely.

Table I summarises the canonical Huffman results for each scenario. The average code length is the weighted mean over all 256 symbols; the compression ratio is defined as 8 / average_code_length.

The highly sparse scenario (FREQS[0x00] = 10000) models early-training gradients where zero bytes dominate. The moderate scenario (FREQS[0x00] = 1000) models mid-training gradients. The dense scenario (FREQS[0x00] = 100) models late-training gradients where the byte distribution is nearly uniform.

The 256-symbol Huffman hardware is designed to compress bfloat16 gradient tensors before gRPC transmission in a distributed LLM training system. Three frequency distributions were defined that model different stages of training, and a canonical Huffman codebook was computed for each using Python.

VI. GRADIENT COMPRESSION PERFORMANCE ANALYSIS

A. Motivation and Scenario Definition

The 256-symbol Huffman hardware is designed to compress bfloat16 gradient tensors before gRPC transmission in a distributed LLM training system. Three frequency distributions were defined that model different stages of training, and a canonical Huffman codebook was computed for each using Python.

The highly sparse scenario (FREQS[0x00] = 10000) models early-training gradients where zero bytes dominate. The moderate scenario (FREQS[0x00] = 1000) models mid-training gradients. The dense scenario (FREQS[0x00] = 100) models late-training gradients where the byte distribution is nearly uniform.

B. Compression Results

Table I summarises the canonical Huffman results for each scenario. The average code length is the weighted mean over all 256 symbols; the compression ratio is defined as 8 / average_code_length.

TABLE I - HUFFMAN COMPRESSION ACROSS GRADIENT SCENARIOS

Scenario	FREQS[0x00]	Avg Code Length (bits)	Compression Ratio
Highly Sparse	10,000	3.314	2.41x
Moderate	1,000	5.908	1.35x
Dense	100	7.862	1.02x

The highly sparse distribution achieves 2.41x compression, driven by the dominant zero byte being assigned a single-bit code. The moderate distribution achieves 1.35x. The dense distribution achieves only 1.02x, confirming Huffman coding provides negligible benefit when all symbols are approximately equally likely.

C. Impact on Gradient Co

mmunication

For a GPT-2 model (124 million parameters, bfloat16 precision), each gradient synchronisation round transmits approximately 248 MB. Table II shows the compressed payload and estimated transfer time on a 1 Gbps gRPC link (baseline: 1984 ms).

TABLE II - GRADIENT TRANSFER IMPACT (GPT-2, 248 MB, 1 Gbps LINK)

Scenario	Compressed Size	Transfer Time	Time Saved	Reduction
Highly Sparse (2.41x)	103 MB	823 ms	1161 ms	58.5%
Moderate (1.35x)	184 MB	1470 ms	514 ms	25.9%
Dense (1.02x)	243 MB	1945 ms	39 ms	2.0%
No compression	248 MB	1984 ms	-	-

Under sparse gradient conditions - the dominant case during early training - FPGA Huffman compression reduces the 248 MB payload to 103 MB, cutting transfer time by 58.5% per synchronisation round. For moderate gradients the saving is 25.9%. For dense gradients the saving is negligible (2.0%).

D. Hardware Verification Across Scenarios

Each scenario was verified end-to-end in Vivado behavioural simulation. The Python scripts regenerate huffman_tables.svh for each distribution; the testbench calls config_encoder_full() and config_decoder_full() at runtime - no RTL modifications are required between scenarios.

In all three cases, symbol_out correctly reproduced the input sequence. The waveforms show the expected increase in stream_valid pulse frequency as average code length grows.

VII. DISCUSSION

A. Design Challenges

Three non-trivial design challenges were encountered and resolved. The first was X-propagation in the encoder: initial implementation placed blocking assignments to module-level signals inside always_ff, causing Vivado xsim to propagate unknown values into the registered bit_count even after a syntactically correct synchronous reset. The root cause was xsim's treatment of module-level blocking-assignment targets as having simulation-time state that is not fully overwritten before non-blocking assignments evaluate. The resolution—the always_comb/always_ff split—is a best-practice SystemVerilog coding style [3] supported across all synthesis and simulation tools.

The second challenge was a bit direction mismatch: the encoder initially packed bits LSB-first while the canonical decoder required MSB-first. This caused the decoder's peek logic to read zeros regardless of stream content. The fix was to redesign the encoder to output acc[63:32], flush via left-shift, and pack with acc |= code << (64 − N − L). The decoder's load logic was simultaneously updated to place incoming words MSB-aligned in the bit window.

The third challenge was a testbench configuration timing bug. dec_table_sel was changed from 1 (base) to 2 (symbol_table) before the clock edge intended to write base[2] = 1, redirecting that write to symbol_table[2]. This left base[2] uninitialised (X), causing unknown index computation for all length-2 codes (A and C) while length-1 codes (B) decoded correctly. The fix was to insert one additional clock cycle before the table_sel transition.

B. Limitations and Future Work

The current implementation has several limitations. First, the encoder lacks a flush mechanism for the final partial 32-bit word; a production system requires an end-of-stream signal and associated flush logic. Second, the testbench tests only a single fixed codebook; a rigorous campaign would include randomised streams, edge cases (single-symbol

alphabets, maximum-length codes), and formal property checking [4]. Third, the system has not been through the full Vivado implementation flow, so the achievable clock frequency and resource utilisation on a target device remain unknown. Finally, the design does not include a mechanism for transmitting the Huffman tree alongside the compressed data, which would be required for a self-contained compression system.

VIII. CONCLUSION

This paper has presented the design and verification of an FPGA-targeted Huffman compression pipeline in SystemVerilog. The system comprises a configurable encoder and a canonical decoder connected via a valid/ready handshake interface. The encoder packs variable-length codes MSB-first into a 64-bit accumulator and outputs 32-bit stream words; the decoder uses three configurable canonical tables and a 64-bit sliding window to reconstruct the original symbol sequence.

Behavioural simulation in Vivado 2024 confirmed correct round-trip operation: the encoder produced the expected word 0x9CE739CE for a repeated A, B, C input sequence, and the decoder successfully reconstructed the original symbols. Three significant design challenges were identified and resolved—an X-propagation defect, a bit direction mismatch, and a testbench configuration timing error—each providing valuable insight into hardware design methodology and simulation-based verification practice.

The project demonstrates that canonical Huffman coding is amenable to efficient hardware implementation with fully dynamic, runtime-configurable codebooks. The foundation established here supports straightforward extension to a full compression system including end-of-stream flushing, codebook transmission, and integration with memory or communication interfaces. With synthesis and implementation, the design is expected to achieve high throughput at modest resource cost on modern FPGA families.

The complete source code for this project - including the SystemVerilog encoder and decoder, the system testbench, and the Python Huffman table generation scripts - is publicly available at: **https://github.com/Heatwave591/FPGA-Acceleration-for-Huffman-Coding**

REFERENCES

[1] D. A. Huffman, "A method for the construction of minimum-redundancy codes," Proc. IRE, vol. 40, no. 9, pp. 1098–1101, 1952.

[2] Canonical Huffman code, Wikipedia. [Online]. Available: https://en.wikipedia.org/wiki/Canonical_Huffman_code

[3] IEEE Std 1800-2017, IEEE Standard for SystemVerilog: Unified Hardware Design, Specification, and Verification Language. IEEE, 2017.

[4] X. Devroey et al., "Formal property checking for digital hardware," IEEE Trans. Comput.-Aided Design, 2020.

[5] D. Salomon and G. Motta, Handbook of Data Compression, 5th ed. Springer, 2010.

[6] ARM Ltd., AMBA AXI4-Stream Protocol Specification, IHI0051B. ARM Holdings, 2023.

[7] B. Parhami, Computer Arithmetic: Algorithms and Hardware Designs, 2nd ed. Oxford University Press, 2010.

[8] S. Hauck and A. DeHon, Eds., Reconfigurable Computing: The Theory and Practice of FPGA-Based Computation. Morgan Kaufmann, 2008.

[9] T. A. Welch, "A technique for high-performance data compression," Computer, vol. 17, no. 6, pp. 8–19, 1984.

[10] L. P. Deutsch, "DEFLATE compressed data format specification version 1.3," RFC 1951, IETF, 1996.